Introdução ao Projeto Orientado pelo Domínio - Atividades

DDD: Por quê?

Atividade:: Analogia

1. Compare e comente como se dá a construção de estradas:
feitas sobre caminhos antigos de carroças planejadas para atender diversas demandas de uma cidade (meio ambiente, fluxo, eficiência, economia, ...)
2. Quais modelos/projetos podem ser pensados para cada situação?
3. Qual dará o melhor resultado aos interessados? Analise os impactos das opções.

A construção de estradas pode seguir duas abordagens: pavimentar caminhos antigos (orgânica) ou planejamento sistemático (projetada). A primeira aproveita trajetos já existentes, mas herda limitações topográficas e de segurança. A segunda considera múltiplas demandas: fluxo, meio ambiente, economia, durabilidade e integração regional.

Modelos para cada situação:

- Caminhos antigos: projetos de requalificação (correções pontuais), intervenção mínima, estudos de tráfego local.
- Planejamento sistemático: projeto geométrico, estudo de viabilidade (técnica, econômica, ambiental), modelagem de tráfego futuro, plano de drenagem e mitigação ambiental.

Melhor resultado e impactos:

- Usuários imediatos: benefício rápido com pavimentação simples, mas a solução pode se tornar obsoleta com aumento de tráfego.
- Sociedade regional: estradas planejadas oferecem maior segurança, durabilidade, eficiência econômica e integração, com menor custo de manutenção a longo prazo.
- Meio ambiente: planejamento inclui controle de erosão, drenagem adequada e compensações ambientais, enquanto a pavimentação simples ignora esses aspectos.
- Poder público: investimento inicial maior no planejamento, mas menor custo total do ciclo de vida; a abordagem orgânica gera custos recorrentes com manutenção e reconstruções.

Em síntese, a abordagem planejada, embora mais custosa e demorada no curto prazo, entrega benefícios duradouros e equilibrados para todos os interessados, superando as limitações da simples pavimentação de caminhos antigos.

DDD: Aplicações

Atividade:: Exemplos práticos

1. Estude os exemplos anteriores e faça um breve resumo sobre seu entendimento dos casos;
2. Identifique conceitos e elementos de DDD citados nesses exemplos;
3. Pesquise e cite outros dois casos de sucesso e aplicação da DDD em sistemas conhecidos.

Resumo dos casos

iFood. O caso mostra a modernização de uma plataforma financeira monolítica, com forte acoplamento, dificuldades de suporte e pressão por mais resiliência, performance e escalabilidade. A solução passou por workshops de Event Storming e modelagem com DDD, o que levou à definição de linguagem ubíqua, bounded contexts, domínios principais e decomposição do monólito em microsserviços com comunicação orientada a eventos.

Amazon / AWS. Nos materiais da AWS, DDD é apresentado como base para arquiteturas compostáveis e para o desenho eficiente de persistência no DynamoDB, usando conceitos como linguagem ubíqua, bounded contexts e aggregates. No caso do data mesh, a AWS afirma explicitamente que a abordagem aplica princípios de DDD às arquiteturas de dados, organizando os dados por domínios e tratando-os como produtos sob responsabilidade dos times.

Uber. A Uber relata que, ao chegar a cerca de 2.200 microsserviços críticos, sentiu na prática os custos da complexidade distribuída e propôs a DOMA (Domain-Oriented Microservice Architecture). A proposta é fortemente inspirada em DDD e em outras arquiteturas bem conhecidas, com o objetivo de manter os benefícios dos microsserviços, mas com organização mais alinhada a domínios e maior autonomia dos times.

Netflix. O caso da Netflix é mais sobre arquitetura hexagonal do que sobre DDD puro, mas a lógica é compatível com o mesmo espírito: preparar o sistema para mudanças, isolar a lógica de negócio e tornar adaptações mais seguras. A própria Tech Blog diz que usou princípios de Hexagonal Architecture para se preparar para mudanças no ecossistema do Netflix Studio.

Spotify. Aqui o foco não é DDD diretamente, mas visualização e comunicação de arquitetura. O time adotou o modelo C4 e um catálogo de software para criar diagramas padronizados, com linguagem comum, entendimento de dependências, ownership e relacionamentos entre componentes. Isso ajuda muito em iniciativas de DDD porque torna as fronteiras e os contextos mais visíveis para diferentes times.

Conceitos e elementos de DDD citados nesses exemplos

Os conceitos de DDD que aparecem de forma explícita ou muito próxima são: linguagem ubíqua, bounded contexts, entidades, agregações / aggregate roots, value objects, repositórios, domínios de negócio, ownership por time, e a colaboração intensa com domain experts/stakeholders via workshops e Event Storming. Nos casos mais arquiteturais, aparecem também ideias complementares como separação por fronteiras claras, desacoplamento, autonomia dos serviços e comunicação entre contextos.

Em resumo prático: o iFood explicita linguagem ubíqua, bounded contexts, agregações e eventos; a AWS explicita linguagem ubíqua, bounded contexts, aggregates e data domains; a Grab explicita bounded context, entities, value objects, aggregate root e repositories; a Uber usa DDD como inspiração para organizar microserviços por domínio; e a Netflix aplica a ideia de separar o núcleo de negócio das adaptações externas via arquitetura hexagonal.

Outros dois casos de sucesso de aplicação de DDD em sistemas conhecidos

Grab. A Grab publicou um caso prático em que usou DDD para reorganizar um código desestruturado em um sistema de integração para parceiros. O artigo mostra claramente o uso de bounded contexts, identificação de entidades e value objects, e a noção de aggregate root para estruturar o domínio.

guardian.co.uk (The Guardian). A InfoQ registra que a plataforma do Guardian foi quase totalmente reconstruída usando princípios de DDD. O relato destaca maior envolvimento de especialistas do domínio e a manutenção de um modelo de domínio profundo e maleável, mesmo com prazos apertados.

Atividade:: Limitações

1. Pesquise casos em que a DDD não foi aplicada com sucesso;
2. Comente as causas apontadas para a falha;
3. Indique qual foi a alternativa para lidar com a situação.

Casos de Insucesso na Aplicação do DDD
O "Projeto Experimental" que Ignorou o Código: Uma empresa tentou adotar DDD com foco exclusivo no design estratégico (definição de domínios e contextos), mas negligenciou a implementação tática no código. As equipes criaram belos diagramas de Bounded Contexts, porém o código permanecia procedural e anêmico, sem refletir a complexidade do domínio modelado.

O "Modelo Incompreensível" para o Negócio: Um esforço para modelar a gestão de TI da Microsoft tentou reutilizar o Common Information Model (CIM) existente. O fracasso ocorreu porque o CIM era um modelo de classes detalhado e técnico, e não um modelo de informações conceitual que pudesse ser discutido e validado com especialistas do negócio. A equipe não conseguiu extrair uma Linguagem Ubíqua a partir dele.

O "Agregado Monstro" em um Sistema Científico: Em um projeto para gerenciar experimentos químicos complexos, a aplicação do DDD levou à criação de um agregado Experiment gigantesco. Ele continha dezenas de milhares de objetos e precisava gerenciar centenas de invariantes ao longo de um ciclo de vida de semanas. Esse agregado tornou-se impraticável para implementar, violando o princípio de consistência transacional e criando um gargalo de desempenho e complexidade.

Causas Apontadas para as Falhas
Ênfase Excessiva em Padrões Técnicos ("Padrões pelo Padrão"): As equipes focaram em implementar o cardápio de padrões do DDD (Agregados, Repositórios, Objetos de Valor) como um fim em si mesmo, em vez de usá-los como ferramentas para resolver problemas de negócio. Isso resultou em um design tecnicamente "bonito", mas desconectado da realidade do domínio . É o chamado Cargo Cult, onde se imitam as formas sem entender a essência.

Disponibilidade Limitada de Especialistas de Domínio: O DDD exige colaboração constante e intensa com especialistas do negócio. Quando esses especialistas não estão disponíveis ou engajados, a modelagem se baseia em suposições, levando a um modelo de domínio incorreto ou genérico.

Aplicação em Contextos Inadequados: Tentar usar DDD em projetos simples (onde um CRUD seria suficiente) ou de curto prazo adiciona complexidade acidental sem trazer benefícios proporcionais . Da mesma forma, insistir nos padrões do DDD em domínios com alta conectividade entre entidades (como no caso do experimento químico) força a criação de agregados enormes e pouco práticos.

Modelo Anêmico e Vazamento de Infraestrutura: Um problema comum é a criação de um "modelo anêmico", onde as entidades são apenas recipientes de dados e a lógica de

negócio transborda para camadas de serviço. Isso é frequentemente agravado pelo vazamento de preocupações de infraestrutura para o domínio, como anotações de banco de dados ou exposição direta de entidades na API.

Alternativas para Lidar com a Situação

Simplificar a Abordagem (para projetos inadequados): Nos casos de projetos simples, a alternativa foi abandonar a complexidade do DDD e adotar uma arquitetura mais direta, como o tradicional CRUD com camadas de apresentação, serviço e persistência. O ganho em simplicidade e velocidade de desenvolvimento superou a perda da "pureza" do design .

Usar Event Sourcing como Porta de Entrada (para o "Agregado Monstro"): Para lidar com a complexidade de agregados gigantes e processos de longo prazo, uma alternativa eficaz é migrar para uma arquitetura baseada em Event Sourcing. Em vez de tentar modelar um agregado que encapsula todo o estado e todas as regras, o sistema passa a armazenar uma sequência imutável de Eventos de Domínio (ex: ExperimentoDesenhado, SubstânciasRecebidas). Isso torna o processo mais natural, rastreável e permite reconstruir o estado a qualquer momento, fragmentando a complexidade.

Adotar uma Arquitetura Hexagonal/limpa (para o Modelo Anêmico): Para combater o vazamento de infraestrutura e o modelo anêmico, a alternativa é reestruturar a aplicação usando os princípios da Arquitetura Hexagonal (ou Ports and Adapters). Isso coloca o domínio no centro, completamente isolado de detalhes externos como bancos de dados e APIs. As dependências apontam para dentro, garantindo que as regras de negócio permaneçam puras e testáveis.

Priorizar a Conversa e a Linguagem Ubíqua (para o "Modelo Incompreensível"): Quando a modelagem falha por ser muito técnica ou distante do negócio, a alternativa é recuar dos diagramas e padrões e focar no básico: conversar com os especialistas. Usar técnicas como Event Storming em workshops colaborativos ajuda a redescobrir o domínio e estabelecer uma Linguagem Ubíqua sólida antes de qualquer codificação.

DDD: Principais Conceitos

Atividade:: Domínio Central

1. Considerando o desenvolvimento de um sistema para um Serviço de Entrega de Comidas, indique quais conceitos poderiam ser considerados para seu Domínio Central;
2. Justifique as motivações para incluir estes elementos;
3. Indique conceitos importantes para esse domínio, mas que ficariam de fora do Domínio Central. Justifique.

Em um sistema de entrega de comidas, o Domínio Central deve concentrar os elementos que realmente diferenciam o negócio e geram vantagem competitiva. Os conceitos mais críticos são a roteirização e otimização de entregas, que define as melhores rotas considerando trânsito e tempo de preparo; o matching inteligente entre entregador e pedido, que aloca a pessoa certa com base em localização e carga; o cálculo dinâmico do tempo de entrega (ETA) , que ajusta previsões em tempo real; e a gestão da disponibilidade real dos restaurantes, que vai além do horário comercial para capturar capacidade de produção e imprevistos. Esses elementos são centrais porque impactam diretamente custo operacional, eficiência da frota, confiança do usuário e taxa de cancelamentos, além de envolverem lógicas complexas e proprietárias que não podem ser resolvidas com soluções genéricas.

Por outro lado, diversos conceitos importantes para o funcionamento do sistema ficam fora do domínio central por não oferecerem diferenciação competitiva. A gestão de usuários (clientes e entregadores) é essencial, mas pode ser tratada com ferramentas prontas de autenticação ou CRUDs simples, já que não há vantagem em inovar nessa área. O processamento de pagamentos, embora crítico, é altamente padronizado e regulado, sendo mais seguro e eficiente integrar com provedores especializados, isolando essa complexidade com uma camada anticorrupção. A gestão de cardápio e catálogo de produtos, apesar de necessária, resume-se em grande parte a operações de CRUD com hierarquias, podendo ser modelada de forma direta como um subdomínio de suporte. Por fim, o sistema de avaliações e comentários funciona bem com uma implementação simples de coleta e exibição, reservando eventuais complexidades analíticas para outro contexto.

Assim, a estratégia de modelagem concentra o máximo investimento no domínio central, onde a riqueza de detalhes e a colaboração com especialistas são cruciais, enquanto os demais subdomínios são tratados de forma pragmática – seja com soluções prontas, seja com modelos simplificados – garantindo que o esforço de engenharia se concentre no que realmente torna o negócio único e competitivo.

Atividade:: Linguagem Ubíqua

Considere o conceito de Voo, no contexto de uma empresa do setor de transporte aéreo.

1. Defina e caracterize os possíveis usos desse termo, levando em conta seu uso por diferentes áreas como Controle de Tráfego, Aquisição de Passagens, Faturamento e Manutenção.
2. Quais seriam os possíveis domínios onde esse termo seria aplicado?

Significados de "Voo" por Área

Controle de Tráfego Aéreo

Definição: O voo é uma entidade operacional em tempo real, identificada por um código (ex: callsign) que percorre uma rota no espaço aéreo, com posição, altitude, velocidade e plano de voo atualizados constantemente.

Características: Preocupação com segurança, separação entre aeronaves, autorizações de decolagem/pouso, condições meteorológicas, e coordenação com torres e centros de controle.

Exemplo de atributos: Número do voo, matrícula da aeronave, origem/destino, posição GPS, altitude, velocidade, plano de voo atualizado.

Aquisição de Passagens (Vendas)

Definição: O voo é um produto comercial disponível para compra, composto por origem, destino, horários, tarifas, assentos disponíveis e classes de serviço.

Características: Foco na disponibilidade, preços dinâmicos, regras de reserva (bagagem, alterações), e integração com sistemas de inventário e reservas.

Exemplo de atributos: Número do voo, aeroportos de origem/destino, horários de partida/chegada, tarifas, assentos livres, tipo de aeronave (para o passageiro).

Faturamento (Contabilidade e Receita)

Definição: O voo é uma unidade de geração de receita e custos, associada a bilhetes vendidos, combustível consumido, taxas de aeroporto, salários de tripulação, etc.

Características: Necessidade de alocar custos e receitas por voo para apuração de resultados, cálculos de comissões, e fechamento contábil.

Exemplo de atributos: Identificador do voo, receita total de passagens, custos operacionais (combustível, taxas), tripulação escalada, horário real de partida/chegada (para rateios).

Manutenção de Aeronaves

Definição: O voo é um evento que impacta o desgaste e a manutenção da aeronave, registrando ciclos de pouso/decolagem, horas voadas, e possíveis incidentes técnicos.

Características: Relacionado à programação de manutenções preventivas, rastreamento de falhas, vida útil de componentes e conformidade com regulamentos.

Exemplo de atributos: Matrícula da aeronave, número do voo, tempo de voo, número de ciclos, relatórios de manutenção associados.

Domínios (Bounded Contexts) Aplicáveis

Cada área define um contexto delimitado onde o conceito de "voo" é modelado de forma específica, com suas próprias regras e interações:

- Domínio de Controle de Tráfego Aéreo (ou Gerenciamento de Tráfego): Modela o voo como uma entidade dinâmica em movimento, com foco em segurança, navegação e comunicação em tempo real. Interage com radares, torres e sistemas de auxílio à navegação.
- Domínio de Vendas e Reservas (Comercial): Trata o voo como um item de catálogo, com disponibilidade, precificação e regras de venda. É o contexto onde passageiros interagem para comprar passagens, escolher assentos e gerenciar reservas.
- Domínio de Faturamento e Controladoria: Enxerga o voo como um centro de custo e receita, essencial para a apuração de resultados financeiros, auditoria e compliance fiscal. Integra-se com sistemas de bilhetagem, folha de pagamento e contabilidade.
- Domínio de Manutenção de Frota: Considera o voo como um evento operacional que gera desgaste e necessidade de intervenções técnicas. É fundamental para o planejamento de manutenções, controle de vida útil de peças e segurança da aviação.

Estudo de caso

Atividade:: Reflexão

1. Estaria a equipe caindo em um armadilha?
2. Houve desvio dos conceitos originais associados a projetos Scrum? (Produtos, Itens de Backlog, Lançamentos, Sprints, ...)
3. O que dizer sobre a linguagem? É clara ou confusa? É consistente? É funcional?
4. Ainda, haverá mais conceitos de suporte a cada conceito nomeado…
5. Caminhamos para uma "Grande Bola de Lama"?
6. Como lidar com isso?

1. Sim, a equipe está caindo em uma armadilha comum: a expansão descontrolada do modelo sem a devida separação em contextos delimitados (bounded contexts). Os conceitos originais associados ao Scrum (Produto, Itens de Backlog, Sprints, etc.) foram gradualmente misturados com funcionalidades periféricas como usuários, fóruns, pagamentos, RH e calendários. Isso gera um modelo monolítico e acoplado, onde regras de negócio distintas se misturam, dificultando a manutenção, a evolução e a clareza conceitual.

2. Houve sim um desvio dos conceitos originais do Scrum, pois os novos requisitos não fazem parte do domínio central de gestão de projetos ágeis. Embora sejam importantes para o produto, eles pertencem a outros domínios (como colaboração, assinaturas, recursos humanos) e deveriam ser modelados em contextos separados, com suas próprias linguagens e regras, comunicando-se com o núcleo Scrum por meio de interfaces bem definidas. A falta dessa separação leva a um modelo inchado e frágil, típico da "armadilha do modelo único" que o Domain-Driven Design busca evitar.

3. É confusa, as interações não são bem delimitadas, e com tanta informação não se sabe pra onde olhar, diminuindo a eficácia do modelo.

5. Sim, a equipe está caminhando perigosamente para uma "Grande Bola de Lama" (Big Ball of Mud), que é um anti-padrão arquitetural onde o sistema cresce sem uma estrutura clara, com dependências emaranhadas, conceitos misturados e ausência de limites definidos. Os sintomas estão claros:

Mistura de conceitos de diferentes domínios: Scrum (produtos, sprints) está sendo embaralhado com colaboração (fóruns, posts), financeiro (pagamentos, planos), RH (equipes, horários) e calendários, tudo no mesmo modelo.

Ausência de limites: Não há separação entre o que é central (gestão ágil) e o que é periférico (assinaturas, suporte, colaboração).

Crescimento orgânico sem direção: Cada novo requisito é simplesmente adicionado ao modelo existente, aumentando o acoplamento e a complexidade.

O resultado previsível é um sistema frágil, difícil de manter, evoluir e compreender, onde uma mudança em uma parte pode quebrar funcionalidades aparentemente não relacionadas.

6. Para lidar com a "Grande Bola de Lama", a equipe precisa adotar uma abordagem de separação em contextos delimitados (bounded contexts). Isso significa reconhecer que o sistema não é um domínio único, mas uma composição de múltiplos domínios – como gestão de projetos Scrum, colaboração, assinaturas, suporte, RH e calendários – cada um com sua própria linguagem, regras e modelo de dados. Esses contextos devem ser rigorosamente separados, comunicando-se apenas por meio de interfaces explícitas como APIs ou eventos, evitando que conceitos de um vazem para outro. Na prática, pode-se começar isolando gradualmente um contexto periférico (como pagamentos) do núcleo Scrum, criando módulos independentes mesmo dentro de um monolito, e evoluir para uma arquitetura mais desacoplada conforme necessário. O objetivo é conter a complexidade, manter a coerência do domínio central e permitir que cada parte do sistema evolua de forma autônoma e sustentável.

Introdução à DDD

Atividade:: Aplicando DDD

1. Repasse o estudo de caso e, com base nas ideias vistas, tente definir quais seriam os elementos pertinentes ao domínio central de um serviço de Transporte de Passageiros com Veículos de Passeio.
2. Aponte quais seriam os outros domínios relacionados ao domínio central.
3. Identifique quais elementos do domínio central estariam ligados aos domínios em seu entorno.
4. Identifique termos comuns a mais de um domínio, mas com significados distintos.
5. Repita o exercício, considerando um sistema de Entregas de Comida.

1) Transporte de Passageiros com Veículos de Passeio

O domínio central seria tudo aquilo que realmente diferencia esse serviço e gera valor competitivo. Em um serviço de transporte de passageiros sob demanda, isso normalmente envolve:

- **Solicitação de corrida**: o passageiro pede um trajeto.
- **Precificação da corrida**: cálculo de valor com base em distância, tempo, tarifa dinâmica, cancelamento, pedágio etc.
- **Matching entre passageiro e motorista**: encontrar o melhor motorista disponível.
- **Gestão do ciclo da corrida**: criada, aceita, em andamento, concluída, cancelada, sem motorista.
- **Cálculo de ETA e roteamento**: previsão de chegada do motorista e do destino.
- **Alocação e disponibilidade de veículos**: saber quem está livre, onde está e em que condição.
- **Regras de cancelamento e no-show**: o que acontece quando o passageiro ou motorista não comparece.
- **Segurança operacional da corrida**: validações essenciais durante a viagem, como compartilhamento de trajeto e detecção de anomalias.

2) Outros domínios relacionados ao domínio central

Ao redor desse núcleo, aparecem domínios de apoio, que são importantes, mas não costumam ser o diferencial principal do negócio:

- Identidade e autenticação
- Cadastro de passageiro e motorista
- Pagamentos e repasses
- Mapas, geolocalização e geocoding
- Notificações
- Suporte ao cliente
- Avaliações e reputação
- Fraude e risco
- Administração e monitoramento
- Promoções e cupons

3) Elementos do domínio central ligados aos domínios do entorno

Aqui vale ligar o núcleo aos seus vizinhos:

- Solicitação de corrida se conecta com identidade/autenticação e notificações.
- Matching entre passageiro e motorista depende de geolocalização, disponibilidade, cadastro de motoristas e, às vezes, fraude/risco.
- Precificação se conecta com pagamentos, promoções e repasses.
- ETA e roteamento se apoiam em mapas, tráfego e geocoding.
- Gestão do ciclo da corrida conversa com notificações, suporte e administração.
- Cancelamento e no-show impactam pagamentos, regras de negócio e atendimento ao cliente.
- Segurança operacional se liga a fraude, monitoramento e reputação.

4) Termos comuns a mais de um domínio, mas com significados distintos

Alguns termos aparecem em vários contextos, mas com sentidos diferentes:

- **Corrida**
    - no domínio de transporte: viagem contratada entre origem e destino;
    - em outro contexto: competição, prova, disputa.
- **Status**
    - na corrida: solicitado, aceito, em andamento, concluído;
    - no pagamento: autorizado, recusado, compensado, estornado.
- **Cancelamento**
    - no transporte: desistência da corrida;
    - no pagamento: reversão ou estorno financeiro.
- **Localização**
    - na corrida: posição atual em GPS;
    - no cadastro: endereço fixo do usuário.
- **Usuário**
    - no app: passageiro;
    - em outro módulo: motorista;
    - em backend: conta autenticada, independentemente do papel.
- **Taxa**
    - na corrida: taxa de deslocamento, tarifa dinâmica, taxa de cancelamento;
    - no financeiro: taxa de processamento, taxa de serviço.

5) Sistema de Entregas de Comida

Agora, no caso de **delivery de comida**, o domínio central muda. O diferencial não é transportar pessoas, mas **orquestrar pedido, preparação, coleta e entrega com qualidade e tempo certo**.

Domínio central

Os elementos mais centrais costumam ser:

- Criação e gestão do pedido
- Associação do pedido ao restaurante
- Acompanhamento do preparo
- Alocação do entregador
- Coleta no restaurante
- Entrega ao cliente
- Cálculo de ETA
- Precificação da entrega
- Gestão de status do pedido
- Regras de substituição, atraso e cancelamento

Em texto corrido: o núcleo é garantir que o pedido saia do cliente, passe pelo restaurante, seja coletado no momento certo e chegue ao destino dentro de uma experiência previsível.

Outros domínios relacionados

Ao redor do core, surgem domínios de suporte como:

- Catálogo de restaurantes e cardápio
- Pagamentos
- Gestão de estoque do restaurante
- Cadastro de clientes, restaurantes e entregadores
- Geolocalização e mapas

- Notificações
- Atendimento ao cliente
- Promoções e cupons
- Fraude e risco
- Avaliações e reputação

Elementos do domínio central ligados ao entorno

- **Pedido** se conecta com **catálogo/cardápio**, **pagamento** e **notificações**.
- **Acompanhamento do preparo** se conecta com o **restaurante**, seu **estoque** e sua **operação interna**.
- **Alocação do entregador** depende de **geolocalização**, **disponibilidade** e, às vezes, **reputação**.
- **Coleta e entrega** se ligam a **mapas**, **rota**, **ETA** e **atendimento ao cliente**.
- **Precificação da entrega** conversa com **pagamento**, **cupons** e **taxas**.
- **Cancelamento e atraso** impactam **suporte**, **reembolso** e **comunicação com o restaurante/cliente**.

Termos comuns com significados distintos

- **Pedido**
    - no delivery: solicitação de comida;
    - no restaurante: pedido interno da cozinha;
    - em outro domínio: ordem de compra ou requisição administrativa.
- **Status**
    - do pedido: recebido, em preparo, pronto, coletado, entregue, cancelado;
    - do pagamento: autorizado, capturado, recusado, estornado.
- **Entrega**

    - no delivery: ato de levar a comida ao cliente;
    - em outro contexto: entrega de documento, pacote ou produto digital.
- **Localização**
    - do cliente: endereço de entrega;
    - do entregador: posição em tempo real.
- **Cancelamento**
    - do pedido: desistência do cliente ou do restaurante;
    - do pagamento: estorno financeiro.
- **Item**
    - no cardápio: prato, bebida, combo;
    - no pedido: linha comprada, com quantidade e observações.